

EfficientMatch : un algorithme taillé pour la convergence rapide en apprentissage semi-supervisé

Erwan Bertrand-Bonvillain

August 2026

Abstract

Les méthodes d'apprentissage semi-supervisé (SSL) se sont considérablement développées au cours de la dernière décennie, notamment dans le contexte de la classification d'images. Elles ont montré qu'il était possible d'atteindre des performances avoisinant celles d'un apprentissage supervisé classique avec seulement quelques dizaines de données étiquetées, sur des benchmarks classiques tels que CIFAR, SVHN ou STL-10. Ces avancées reposent toutefois sur un protocole expérimental standardisé et coûteux, imposant un budget de calcul fixe de 2^{20} itérations ainsi qu'un ensemble d'hyperparamètres communs destinés à garantir une comparaison équitable entre méthodes. Ce protocole a orienté la recherche vers un objectif unique : la performance asymptotique, atteinte au prix d'un temps d'entraînement considérable. Nous soutenons que cet objectif occulte une dimension pourtant essentielle à l'applicabilité pratique des méthodes SSL : leur efficacité de convergence à court terme. Nous présentons EfficientMatch, une méthode conçue non pas pour maximiser la performance finale après un entraînement long, mais pour converger rapidement vers une performance élevée, dans l'optique d'une meilleure compréhension des mécanismes responsables de l'efficacité des méthodes SSL et d'un déploiement facilité dans des contextes à ressources de calcul limitées.

1 Introduction

L'apprentissage semi-supervisé (SSL) en classification d'images a connu des progrès marquants depuis le pseudo-étiquetage [7], jusqu'à des méthodes comme MixMatch [2], FixMatch [9] ou FlexMatch [14], qui approchent aujourd'hui les performances d'un apprentissage supervisé classique avec seulement quelques dizaines de labels. Ces gains reposent sur un protocole devenu standard [12] : budget fixe de 2^{20} itérations, architecture WideResNet [13], hyperparamètres partagés – garantissant la comparabilité, mais cristallisant la performance asymptotique comme unique critère de succès et imposant un coût de calcul important [12].

Plusieurs travaux ont déjà interrogé cette hypothèse implicite : FlexMatch [14] montre qu’une fraction du budget standard suffit à dépasser les performances finales de FixMatch, et Fast FixMatch [4] réduit directement le calcul d’entraînement via un curriculum de taille de batch (CBS). Ce corpus reste cependant fragmenté : chaque méthode définit sa propre notion d’efficacité, sans protocole d’évaluation partagé ni méthode conçue *a priori* pour ce régime plutôt qu’évaluée *a posteriori* sur un budget tronqué.

Notre contribution est double : (i) un protocole d’évaluation réduit (2^{17} itérations, schedule recalé, métriques d’efficacité indépendantes du hardware) et (ii) **EfficientMatch**, qui combine la structure FixMatch à un canal de Mixup [15] appliqué exclusivement sur les pseudo-étiquettes filtrées par confiance – contrairement à MixMatch/ReMixMatch [2, 3] qui mélangent sans filtrage. Ce canal, à coût de calcul marginal (§3.1), vise à concilier la stabilité en faible labellisation des méthodes à seuillage avec l’exploitation efficace des labels propre au Mixup. Nous évaluons cette hypothèse sur CIFAR-10 et un jeu hors distribution (PathMNIST [yang2023medmnist]), avec une étude de robustesse architecturale et une analyse de complémentarité avec Fast FixMatch en annexe.

2 Related Work

Mécanismes SSL. Les méthodes récentes combinent quatre leviers : des politiques d’augmentation (RandAugment [5], CTAugment [3]), le Mixup [15] (interpolation convexe favorisant un comportement linéaire du modèle [chidambaram2021provable]), le seuillage de confiance contre le biais de confirmation [1] (fixe dans FixMatch [9], adaptatif par classe depuis FlexMatch [14]), et la cohérence faible/forte – héritée des travaux antérieurs de régularisation par cohérence [6, 10, 8], combinée pour la première fois au seuillage dans FixMatch. MixMatch [2] et ReMixMatch [3] exploitent le Mixup sans seuillage explicite, ce qui limite leurs performances en très faible labellisation (chiffre à sourcer précisément, cf. §3) malgré une exploitation plus directe du signal étiqueté.

Protocole standardisé et son coût. Le protocole hérité de FixMatch (budget fixe, architecture partagée) a été formalisé à plus large échelle par USB [12], dont les auteurs motivent leur travail par le coût computationnel et environnemental de l’entraînement *from scratch*. Ce protocole cristallise implicitement la performance de fin d’entraînement comme unique objectif de conception.

Efficacité de convergence. FlexMatch [14] dépasse les performances finales de FixMatch en une fraction du budget standard sur CIFAR-100 et argue explicitement contre la comparaison sur seule médiane de fin d’entraînement. Fast FixMatch [4] réduit le calcul d’entraînement d’un facteur $2.1\text{--}3.4\times$ à taux d’erreur égal via un curriculum de taille de batch non étiqueté (CBS), et DoubleMatch [11] réduit les temps d’entraînement en exploitant plus densément les données non étiquetées. Ces travaux partagent notre constat – l’efficacité de convergence est un axe distinct de la performance asymptotique – mais restent

des variantes accélérées de FixMatch, sans protocole d'évaluation commun ni canal de régularisation additionnel comme celui proposé ici (cf. discussion, §??).

3 Method

3.1 Notations

Batch étiqueté $\mathcal{X} = \{(x_b, y_b)\}_{b=1}^B$, batch non étiqueté $\mathcal{U} = \{u_b\}_{b=1}^{\mu B}$. $\alpha(\cdot)$: augmentation faible ; $\mathcal{A}(\cdot)$: augmentation forte (RandAugment). Pour $u_b^w = \alpha(u_b)$, on note $q_b = p_m(y \mid u_b^w; \theta)$, $\hat{q}_b = \arg \max(q_b)$, et le masque de confiance $m_b(\tau) = \mathbb{1}(\max(q_b) \geq \tau)$ (τ fixe dans FixMatch/EfficientMatch, adaptatif par classe dans FlexMatch [14]). L'opérateur de Mixup [15] est $\text{Mix}_\lambda((x_i, p_i), (x_j, p_j)) = (\lambda x_i + (1 - \lambda)x_j, \lambda p_i + (1 - \lambda)p_j)$, $\lambda \sim \text{Beta}(\alpha_m, \alpha_m)$.

3.2 EfficientMatch

EfficientMatch combine la perte FixMatch classique $\mathcal{L}_s + \lambda_u \mathcal{L}_u$ à un canal de Mixup filtré $\mathcal{L}_{\text{mix}}$, appliqué sur l'ensemble concaténé-mêlé de $\mathcal{X}$ et des pseudo-étiquettes de $\mathcal{U}_w$, pondéré par le produit des masques de confiance des deux éléments de chaque paire mixée (masque dur ; les exemples étiquetés reçoivent un masque = 1 par convention) :

$$\mathcal{L}_{\text{mix}} = \frac{1}{B + \mu B} \sum_k m_k \cdot m'_k H(\tilde{p}_k, p_m(y \mid \tilde{x}_k; \theta)), \quad \mathcal{L} = \mathcal{L}_s + \lambda_u \mathcal{L}_u + \lambda_{\text{mix}} \mathcal{L}_{\text{mix}}.$$

Le pseudo-code complet est donné en Algorithme ?? (Annexe ?? – ou en corps si la place le permet, à trancher en dernier).

Différenciation. Contrairement à MixMatch/ReMixMatch, le signal injecté dans $\mathcal{L}_{\text{mix}}$ n'est jamais construit à partir d'un pseudo-label jugé peu fiable par le modèle. Contrairement à FixMatch/FlexMatch, ce canal ajoute une régularisation supplémentaire sans passage réseau additionnel (le Mixup opère sur les représentations déjà calculées), donc à coût marginal quasi nul – hypothèse centrale testée en §4 (RQ1). Une discussion détaillée par rapport à chaque méthode est fournie en Annexe ??.

4 Experiments

RQ1. EfficientMatch converge-t-il plus vite, à budget de calcul égal, que FixMatch et FlexMatch ? **RQ2.** Reste-t-il compétitif en très faible labellisation, où MixMatch échoue historiquement ? *Questions secondaires (généralisation hors distribution, dépendance architecturale, complémentarité avec Fast Fix-Match) traitées en Annexe ??.*

[Figure double panneau : (a) accuracy vs. itérations, (b) accuracy vs. FLOPs
– une courbe par méthode, moyenne $\pm$ écart-type sur 3 seeds]

Figure 1: Convergence d’EfficientMatch comparée à MixMatch, FixMatch et FlexMatch sur CIFAR-10.

Table 1: Itérations/FLOPs pour atteindre [X%] de l’accuracy asymptotique de FixMatch, AUC normalisée, et accuracy finale au régime de faible labellisation (RQ1+RQ2 combinées). Moyenne $\pm$ écart-type sur 3 graines.

Méthode	Itérations $\downarrow$	FLOPs $\downarrow$	AUC $\uparrow$	Acc. finale
MixMatch	–	–	–	–
FixMatch	–	–	–	–
FlexMatch	–	–	–	–
EfficientMatch	–	–	–	–

Protocole. Budget plafond de 2^{17} itérations (schedule cosine recalé, cf. §2), CIFAR-10, WideResNet-28-2 [13], régime de faible labellisation [valeur], 3 graines aléatoires. Chaque run est interrompu par un critère d’arrêt anticipé (cf. paragraphe suivant) plutôt que systématiquement mené à 2^{17} .

Critère d’arrêt anticipé. La décroissance du schedule cosine implique mécaniquement une réduction des mises à jour de poids – et donc de la variation d’accuracy – en fin d’entraînement, cohérent avec les observations de Zhang et al. [14] sur le protocole standard. Nous interrompons chaque run lorsque la pente d’une régression linéaire locale de l’accuracy EMA, sur une fenêtre de [5] évaluations consécutives, passe sous un seuil $\epsilon = [10^{-4}]$ – critère préféré à un simple comptage d’évaluations sans amélioration pour limiter la sensibilité au bruit propre au régime de faible labellisation. Le seuil ϵ et la taille de fenêtre sont calibrés sur des runs pilotes (Annexe A) et appliqués identiquement à toutes les méthodes comparées, pour ne pas introduire de biais favorable à l’une d’entre elles. La courbe accuracy-vs-itérations de chaque méthode est complétée par report de la dernière valeur EMA observée jusqu’à 2^{17} , afin que l’AUC (Area Under Curve) reste calculable et comparable entre méthodes arrêtées à des moments différents. Le calcul de l’AUC intègre le report de dernière valeur décrit ci-dessus pour les runs interrompus avant 2^{17} itérations.

Paragraphe d’analyse unique couvrant RQ1 et RQ2 conjointement (Fig. 1, Tab. 1) : EfficientMatch converge-t-il plus vite que FixMatch/FlexMatch (RQ1) tout en évitant la chute de MixMatch en faible labellisation (RQ2) ? Discuter les trois cas de figure (hypothèse confirmée / partielle / infirmée) en s’appuyant sur le mécanisme de filtrage (§3.2).

4.1 Limitations

L’ensemble des expériences repose sur une unique GPU A4000, ce qui restreint le nombre de configurations testées (un budget de labels, trois graines) et repousse en Annexe ?? les questions de généralisation hors distribution, de dépendance architecturale et de complémentarité avec les méthodes d’efficacité concurrentes. Fast FixMatch [4], discuté en §2 comme travail le plus proche en objectif, fait l’objet d’une comparaison expérimentale limitée pour la même raison (Annexe ??).

Le critère d’arrêt anticipé, bien que calibré identiquement pour toutes les méthodes, introduit une source additionnelle de variabilité entre graines (l’itération d’arrêt effective diffère d’un run à l’autre) – discutée en Annexe A.

5 Conclusion

References

- [1] Eric Arazo et al. “Pseudo-labeling and confirmation bias in deep semi-supervised learning”. In: *2020 International joint conference on neural networks (IJCNN)*. IEEE. 2020, pp. 1–8.
- [2] David Berthelot et al. “Mixmatch: A holistic approach to semi-supervised learning”. In: *Advances in neural information processing systems* 32 (2019).
- [3] David Berthelot et al. “Remixmatch: Semi-supervised learning with distribution alignment and augmentation anchoring”. In: *arXiv preprint arXiv:1911.09785* (2019).
- [4] Yu Chen et al. “Fast FixMatch: Faster Semi-Supervised Learning with Curriculum Batch Size”. In: *European Conference on Computer Vision (ECCV)*. 2024.
- [5] Ekin D. Cubuk et al. *RandAugment: Practical automated data augmentation with a reduced search space*. 2019. arXiv: 1909.13719 [cs.CV]. URL: <https://arxiv.org/abs/1909.13719>.
- [6] Samuli Laine and Timo Aila. “Temporal ensembling for semi-supervised learning”. In: *arXiv preprint arXiv:1610.02242* (2016).
- [7] Dong-Hyun Lee et al. “Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks”. In: *Workshop on challenges in representation learning, ICML*. Vol. 3. 2. Atlanta. 2013, p. 896.
- [8] Takeru Miyato et al. “Virtual adversarial training: a regularization method for supervised and semi-supervised learning”. In: *IEEE transactions on pattern analysis and machine intelligence* 41.8 (2018), pp. 1979–1993.
- [9] Kihyuk Sohn et al. “Fixmatch: Simplifying semi-supervised learning with consistency and confidence”. In: *Advances in neural information processing systems* 33 (2020), pp. 596–608.

- [10] Antti Tarvainen and Harri Valpola. “Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results”. In: *Advances in neural information processing systems* 30 (2017).
- [11] Erik Wallin et al. “Doublematch: Improving semi-supervised learning with self-supervision”. In: *2022 26th International Conference on Pattern Recognition (ICPR)*. IEEE. 2022, pp. 2871–2877.
- [12] Yidong Wang et al. “USB: A Unified Semi-supervised Learning Benchmark for Classification”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 3938–3961.
- [13] Sergey Zagoruyko and Nikos Komodakis. “Wide Residual Networks”. In: *arXiv preprint arXiv:1605.07146* (2016).
- [14] Bowen Zhang et al. “FlexMatch: Boosting Semi-Supervised Learning with Curriculum Pseudo Labeling”. In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021, pp. 18408–18419.
- [15] Hongyi Zhang et al. “mixup: Beyond empirical risk minimization”. In: *arXiv preprint arXiv:1710.09412* (2017).

A Calibration du critère d’arrêt anticipé

[Figure : courbes accuracy vs. itérations sur runs pilotes (budget réduit, une seed), une par méthode, avec la pente locale superposée en second axe pour justifier visuellement le choix de ϵ et de la taille de fenêtre. Texte d’accompagnement : montrer que le critère détecte le plateau sans couper de vraie phase de progression tardive (risque discuté en §4).]

B Hyperparamètres

Table 2: Hyperparamètres utilisés dans les expériences. Les valeurs communes suivent le protocole standard [9, 14], à l’exception du budget d’itérations K , recalé sur notre protocole réduit, et du critère d’arrêt anticipé propre à ce travail.

Hyperparamètre	Valeur
<i>Communs à toutes les méthodes</i>	
Optimiseur	SGD, momentum 0.9, Nesterov
Learning rate initial η_0	0.03
Schedule	$\eta = \eta_0 \cos(\frac{7\pi k}{16K})$, $K = 2^{17}$
Weight decay	5e-4 (CIFAR-10, PathMNIST) / 1e-3 (CIFAR-100)
Batch size étiqueté B	64
Ratio non étiqueté μ	7
Seuil de confiance τ	0.95
Poids perte non supervisée λ_u	1
Momentum EMA (évaluation)	0.999
Budget plafond d’itérations K	2^{17} (1024 itérations/epoch)
Graines aléatoires	3
<i>Critère d’arrêt anticipé</i>	
Taille de fenêtre (évaluations)	[5]
Seuil de pente ϵ	[10^{-4}]
Fréquence d’évaluation	512 itérations
<i>Spécifiques au Mixup (MixMatch, EfficientMatch)</i>	
α_m (paramètre Beta)	0.75 (à confirmer après ablation, Annexe ??)
λ_{mix} (EfficientMatch uniquement)	à fixer après ablation (Annexe ??)
<i>Spécifiques à Curriculum Batch Size [4]</i>	
Paramètre α (schedule B-EXP)	0.7
Taille de batch minimale	[8]
<i>Architectures</i>	
Étude principale (RQ1–RQ2)	WideResNet-28-2 [13]
Étude de robustesse (Annexe ??)	ResNet-18 [he2016deep], CIFAR-10 uniquement